The safety guarantees provided by transactions are often described by the well-known acronym "ACID", which stands for:

Atomicity - if a transaction was aborted, the application can be sure nothing was changed, so it can be safely retried

Consistency - idea that you have certain statements about your data that must always be true

Isolation - concurrently executing transactions are isolated from eachother: they cannot step on each others toes

Durability - Once a transaction has been committed to the DB, any data it has written will not be forgotten

Single Object:

- atomicity can be implemented using a log for crash recovery
- isolation can be implemented using a lock on each object

Weak Isolation Levels:

Concurrency issues (race conditions) come into play when one transaction reads data that is concurrently modified by another transaction, or when two transactions try to simultaneously modify the same data

DB's have tried to hide the concurrency issues from application developers by providing transaction isolation; In theory this should

aka

make your life easier: serializable isolation means the DB guarantees transactions have the same effect as if they ran serially

However serializable isolation has a performance cost and DB's dont want to pay that price so they use weaker levels of isolation which protect against some concurrency issues, but not all

Several commonly used weak isolation levels:

1) Read Committed

- Most basic level

Two gurantees:

transaction has r/w some data to DB but the transaction has not yet committed

1. When reading from DB, only see data that has been committed (no dirty reads)

Why?

- if a transaction needs to update several objects, a dirty read means that another transaction may see some of the updates but not others.
- a transaction may see data that is later rolled back after an abort

2. When writing to the DB, you only overwrite data that has been committed (no dirty writes)

Implementation:

- Prevent dirty writes by using row-level locks: when a trans. wants to modify an object (row/doc), it must first lock the object. Then it must hold the lock until the trans. has either committed or aborted.
- Prevent dirty reads with the following approach: for every object that is written, the DB remembers both the old committed value and the new value set by the trans. that currently holds the lock. While trans. ongoing any other trans. that read the object are simply given the old value

(this is acceptable under read committed isolation level)

read skew - when a transaction reads multiple related data items, but because other transactions commit changes in between those reads, the transaction sees a mixed, inconsistent snapshot of the database that never existed at a single point in time.

↓ an example of non-repeatable reads

non-repeatable read - when a transaction reads the same row twice and gets different values because another transaction updated and committed the row in between.

This temporary inconsistency is not acceptable for backups, analytic queries and integrity checks

2) Snapshot Isolation and Repeatable Read

Snapshot isolation - idea is that each transaction reads from a consistent snapshot — transaction sees all the data that was committed to the database at the start of the transaction

is the most common solution ensure **repeatable read**

Implementation:

- **Key principle** - reads do not require locks; readers never block writers and writers never block readers

to implement this, need to keep multiple versions of the same object side by side. This technique is called **multi-version concurrency control (MVCC)**

MVCC is implemented by giving each transaction a monotonically increasing ID, and tagging each write with the transaction's ID (created_by and deleted_by field)

initially empty. rows are marked for deletion and are actually deleted when 100% sure the value is unreachable

An update is internally translated into a delete and create. Old value marked for deletion. Updated value is created.

hiding allows DB to present consistent snapshot

An object is visible to a transaction if:

1. At the time when the readers transaction started, the transaction that created the object already committed
2. The object is not marked for deletion, or if it is, the transaction that requested deletion had not yet committed at the time when the reader's transaction started

Preventing Lost Updates:

- have only covered guarantees of what a read-only transaction can see in the presence of concurrent writes

lost update problem – can occur if the application reads some value from the database, modifies it, and writes back the modified value

(a **read-modify-write** cycle)

↓

if two transactions do this concurrently, one of the modifications can be lost because the second write does not include the first modification

(second write **clobbers** the first one)

Variety of Solutions because its a common problem:

1. **Atomic write operations**
   - remove the need to implement read-modify-write cycles
   - usually implemented by taking an exclusive lock on the object so no other transaction can read it until the update has been committed (technique known as **cursor stability**)

2. **Explicit Locking**
   - explicitly lock objects that are going to be updated. This allows the application to perform a read-modify-write cycle, while other transactions that want to concurrently read are forced to wait

3. Automatically detecting lost updates

- let transactions execute in parallel and if the transaction manager detects a lost update, then the transaction is aborted and forced to retry
- Most DB can efficiently perform this check in conjunction with snapshot isolation

4. Compare and set

- DB's that dont provide transactions may provide atomic compare-and-set operation
- avoids lost updates by allowing an update to happen only if the value has not changed since you last read it, if it has the update has no effect and must be retried

5. Conflict resolution and replication

- another dimension added to preventing lost updates on replicated databases
- cannot use techniques that rely on locks / compare-and-set
- common approach is to allow concurrent writes and use application code to resolve and merge conflicting versions
- atomic operations can work well especially if the operations are commutative

Write Skew and Phantoms :

Write skew - Can occur if two transactions read the same objects and then update some of those objects

↓

a generalization of the lost updates problem.

Examples of situations it can occur :

- Meeting room booking system
- Multiplayer game
- Claiming a username

↓ follow a similar pattern

1. SELECT query is used to check whether some requirement is satisfied by searching for rows that match some condition

   (no existing bookings for that room at that time, username isn't already taken)

2. Depending on result of query, application code decides what to do next

3. If app. decides to continue, it makes a write (INSERT, UPDATE, or DELETE) and commits transaction

The effect of this write changes the precondition of the decision in step 2

(the meeting room is now booked for that time, that username is now taken)

phantoms - the effect where a write in one transaction changes the result of a search query in another transaction

↓

Snapshot isolation prevents this in read-only queries, but in read-write transactions it can lead to write skew

Serializability:

Serializable isolation - guarantees that parallel executing transactions will have the same end result as if they had executed one after another AKA prevents all possible race conditions (strongest isolation level)

Most DBs use one of the three techniques to provide serializability:

1. Actual Serial Execution (Redis)
   - remove concurrency, only execute one transaction at a time in serial order on one thread
   - possible by keeping all data in memory and/or the fact that OLTP transactions are very short

- To avoid network communication time, these systems dont allow interactive multi-statement transactions, rather the application must submit the entire transaction code to the DB ahead of time as a stored procedure

  ↓

  is basically code running on the DB avoiding the process of making multiple read requests that get necessary data for application logic

Summary: transactions small and fast, active dataset can fit in memory, write throughput must be low enough to be handled on a single CPU core otherwise transactions need to be partitioned without requiring cross-partition coordination

2. Two-Phase Locking (2PL) (MySQL)

- transactions can concurrently read an object, but as soon as one wants to write exclusive access is required:
  - transaction A has read an object, and transaction B wants to write to that object. B must wait until A commits/aborts
  - transaction A has written to an object, and transaction B wants to read it. Then, B must wait until A commits/aborts
- locks can be in shared mode or exclusive mode
- locks are used as follows:
  - shared lock for reading which multiple transactions can hold

simultaneously

- exclusive lock for writing, no other transaction may hold a lock on that object at the same time
- to read then write, a transaction can upgrade its lock. Must wait for other transactions to finish before getting exclusive lock
- Must hold the lock until transaction commits/aborts

- performance bad as queues may form due to even just a single long transaction —— unstable latencies

- use predicate locks to prevent phantoms; a shared/exclusive lock that belongs to all objects that match some search condition (→ meeting rooms example)

  ↓

  select ... where ... → all results are locked

- predicate locks have bad performance because checking for matching locks is time consuming, instead use index-range locks (aka next-key locking) which is a simplified approximation of predicate locking

  ↓

  make it match a greater set of objects; predicate lock on booking room 123 from noon - 1 pm, just lock bookings for 123 at anytime or lock booking from noon - 1 pm on all rooms.

3. Serializable Snapshot Isolation

- relatively young, but has the best performance out of the three
- optimistic concurrency control, meaning instead of blocking if something dangerous can happen, transactions continue anyway in the hope that everything will turn out fine (unlike 2PL which is pessimistic)
- When a transaction wants to commit, DB checks that nothing bad happened, if so the transaction is aborted and must be retried. Only transactions that executed serializably are allowed to commit.
- DB knows if a query result might have changed by:
  - Detecting reads of a stale MVCC object version (uncommitted write occured before the read)
  - Detecting writes that affect prior reads (the write occurs after the read)